

Agent Mesh Observatory & Governor

Concept & V0 Product Spec (Updated Pivot: Tool/Skill Authorization + Agent Forensics) — Dec 23, 2025

1. One-liner

A cross-runtime observability, policy enforcement, and forensics layer for organizations running many internal AI agents and skills. It maps how agents call tools and each other, enforces fine-grained tool/skill permissions, and produces incident-grade evidence bundles when things go wrong.

2. Problem

- **No global visibility:** teams ship agents across frameworks/vendors; nobody can see the true interaction graph.
- **New risk surface:** prompt injection and tool abuse turn the agent into a privileged execution layer (web, SQL, tickets, code, payments).
- **Weak governance:** rules are ad-hoc (best practices, scattered allowlists). Enforcement is inconsistent across runtimes.
- **Postmortems are painful:** traces exist, but assembling an incident timeline + evidence bundle across stacks is still manual.

3. Product overview

3.1 What it does (V0)

- Instruments tool calls and agent-to-agent calls via lightweight SDKs and/or an HTTP proxy.
- Builds a live **agent mesh graph** (agents/skills/tools/systems as nodes; calls as edges) with stats (calls, errors, latency, cost).
- Evaluates **tool/skill permission policies** via a central Policy Decision Point (PDP) and enforces decisions at Policy Enforcement Points (PEPs).
- Provides an **incident flight recorder**: timeline view, path replay, and one-click export of an evidence bundle (graph snapshot + key events + metrics + policy context).
- Supports containment guardrails: per-tool/agent QPS & concurrency limits, kill switches, and break-glass overrides (time-bound).

3.2 Who it's for

- **Primary buyer:** Head of Platform/Infra, Head of AI Platform/Enablement; often co-bought with AppSec/GRC as the program matures.
- **Primary users:** platform/SRE teams, agent framework owners, security engineers, incident responders.

4. Why now

- At 1–2 agents, local logs work. At 20–100+ agents across teams and vendors, emergent behavior and security incidents become inevitable — and agents increasingly call real tools on real systems.

5. Differentiation

Framework vendors may expose logs/metrics for their own runtimes. This product is explicitly **cross-runtime** and focused on: (a) consistent permission enforcement across heterogeneous stacks, and (b) incident-grade evidence production.

- **Cross-runtime**: unify OpenAI/Anthropic/local LLM agents, homegrown orchestrators, Claude Skills/tooling, and arbitrary tools into one graph and one policy plane.
- **Not just a gateway**: fine-grained authorization decisions with rationale + obligations, plus immutable-ish audit trails.
- **IR-ready**: “what happened” and “why” export, not only dashboards.

6. V0 goals & non-goals

6.1 Goals (V0)

- Live interaction graph of agents/skills/tools/systems; filter by env/team/time.
- Per-node/per-edge stats: calls, errors, latency, cost estimates.
- Detect basic pathologies: loops, excessive depth, and cost/call spikes.
- **Semantic allow/deny** for tool/skill calls based on policy (resource + parameters + environment).
- Audit logs: policy versioning, decision logs with rationale codes, and who-changed-what config history.
- Incident timeline + evidence bundle export (JSON + human-readable summary).

6.2 Non-goals (V0)

- No automatic agent auto-tuning or prompt rewriting.
- No deep content inspection by default (store summaries/hashes; raw prompts are optional and customer-controlled).
- No “perfect” prompt-injection prevention claim; we focus on enforceable controls + evidence.
- No multi-tenant SaaS hardening for ultra-regulated environments in V0 (start self-hosted or single-tenant).

7. Core capabilities

7.1 Telemetry & integration

Integration options:

- SDK hooks/wrappers (Python, Node/TypeScript) that wrap tool calls and emit events.
- Optional language-agnostic HTTP proxy for tool calls where SDK integration is hard.

Canonical event schema (simplified fields):

- **timestamp**
- **trace_id, span_id, parent_span_id**
- **actor_type, actor_id** (agent/skill/tool/system)
- **target_type, target_id**
- **operation** (tool name/op) + **status/error_type**
- **input_summary / output_summary** (redacted or hashed)
- **latency_ms, cost_estimate**
- **env** (prod/stage/dev) + **metadata** (team, tenant, tags)

7.2 Policy plane (PDP/PEP)

- **PDP**: evaluates policies and returns decisions + rationale + obligations.

- **PEPs**: SDK wrapper/proxy that blocks, constrains, or allows calls based on PDP output.
- Policies express: identity, tool, resource, parameters/constraints, environment, time-bound grants, approvals.

MVP policy examples:

- Allow ``sql.query`` in prod only for read-only queries and max 100 rows.
- Allow ``web.fetch`` only for allowlisted domains; deny downloads of executables.
- Allow ``mcp.tool:send_email`` only in staging; require approval in prod.

7.3 Forensics / flight recorder

- Incident timeline: correlate traces, show key decisions, tool calls, anomalies, containment actions.
- Path replay: reconstruct the chain of calls with redacted summaries and policy context.
- Evidence bundle export: graph snapshot + event subset + policy versions + decision rationales.

8. Architecture sketch (V0)

- **Ingest API**: receives telemetry events from SDKs/proxy.
- **Event store**: Postgres initially (partitioned tables) with room for time-series later.
- **Graph builder**: builds/updates caller→callee relationships + aggregates stats.
- **PDP service**: evaluates policies, writes decision logs.
- **Admin/API**: serves graph, metrics, policies, incident artifacts to the UI.
- **Web UI**: graph view, metrics, policy editor, incident timeline, export action.
- **Optional proxy**: enforcement + telemetry in one place for HTTP-based tools.

9. First customer profile (for V0)

- 200–2000 person company with multiple teams building internal agents/skills.
- Using 2+ LLM vendors/frameworks, with real tool integrations (SQL, web, ticketing, code).
- Already worried about spend, reliability, and the security/governance story.

10. Success criteria (V0)

- Within 1–2 weeks, the customer finds at least 1 surprising dependency/hotspot from the graph.
- They prevent/contain at least 1 risky tool action using policy (blocked or constrained call).
- They export an evidence bundle for a simulated incident that a security/IR stakeholder accepts.

Next: V1 can add richer detection, policy-as-code integrations, richer connectors, and deeper enterprise hardening.

Document owner: Sam (with agents). Last updated: Dec 23, 2025.